01 / LAB OVERVIEW

SQL Injection

HackerDNA / Authentication bypass and SQLite data extraction

English edition
Visual presentation: COMMANDO. Based on the supplied walkthrough and its original screenshots.

Lab information

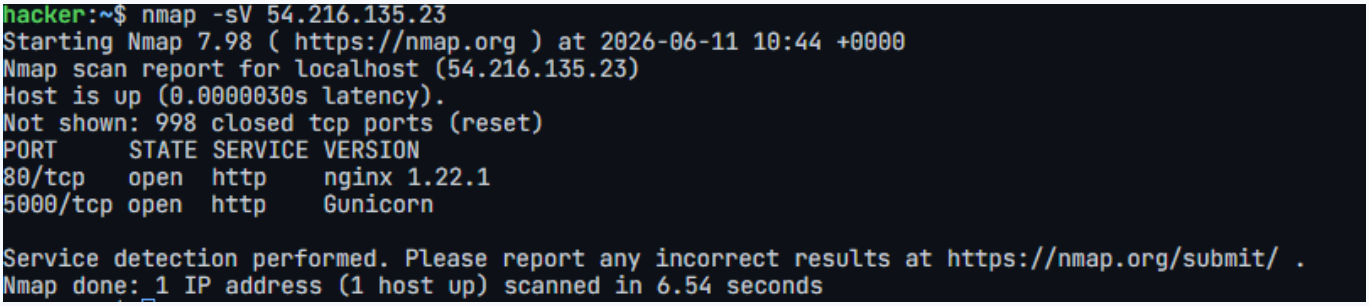
Platform: HackerDNA **Difficulty:** Easy **Flags:** 1
Category: SQL Injection
Lab: hackerdna.com/pt-br/labs/sql-injection/flags

Objective

Identify and exploit a SQL injection vulnerability to access application data without authorization and retrieve the challenge flag.

Service enumeration

```
nmap -sV 54.216.135.23
```



01. Original Nmap output from the documented lab instance.

The scan shows HTTP services on **80/TCP** and **5000/TCP**, identified as Nginx and Gunicorn. The original prose called port 5000 a VPN service; the screenshot identifies it as HTTP.

Methodology

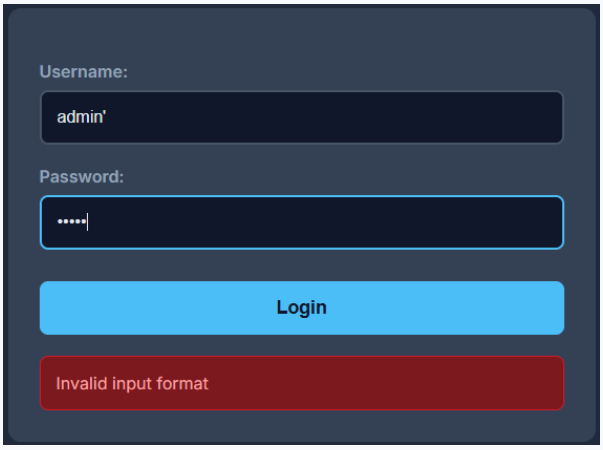
Enumerate services, locate input fields, test for SQL injection, determine the query shape, enumerate the database structure, and extract the flag. The recorded IP belongs to the original lab session.

02 / INPUT TESTING

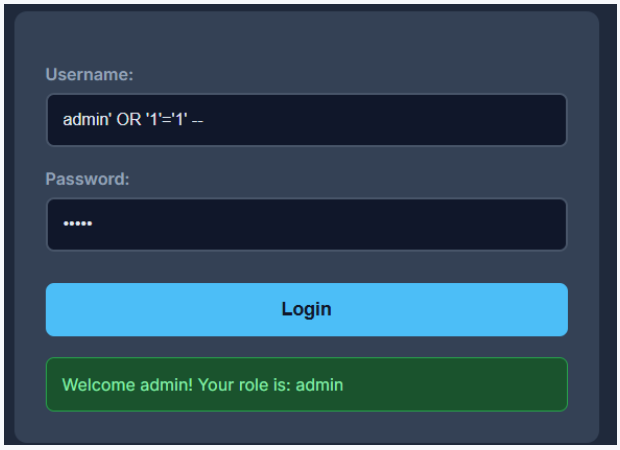
Testing the login form

Observed responses to SQL expressions

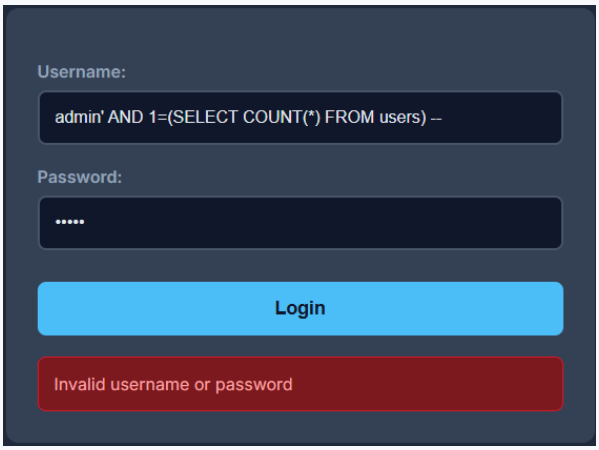
Testing focused on the username field. A quote produced an input-format error, while an OR expression returned an administrator welcome message. Further probes produced mixed results.



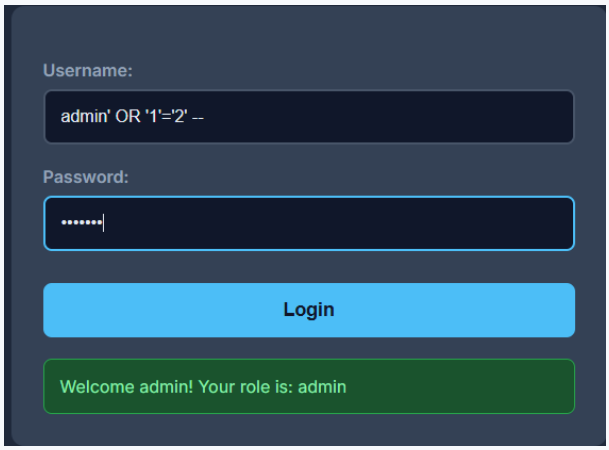
02. Quote probe: input-format error.



03. True OR expression: admin welcome.



04. COUNT probe: invalid credentials.



05. False OR expression: admin welcome.

Interpret the evidence carefully

Both the true and false OR examples display successful login. They therefore do not establish a clean true/false response difference. The underlying query is not shown; the successful UNION tests on the next page provide clearer evidence of injection.

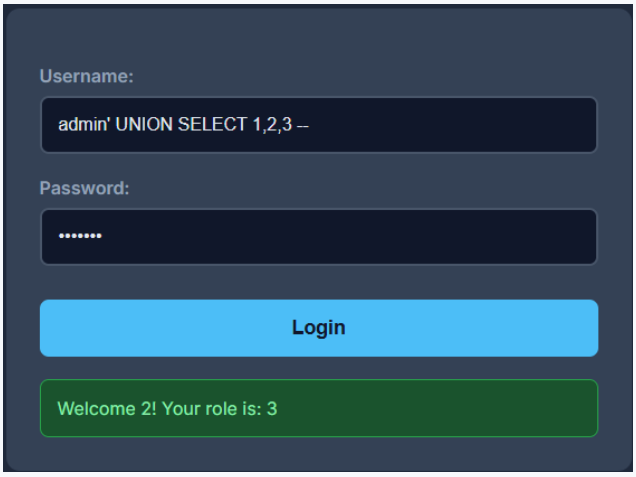
03 / UNION-BASED EXTRACTION

Query shape and engine

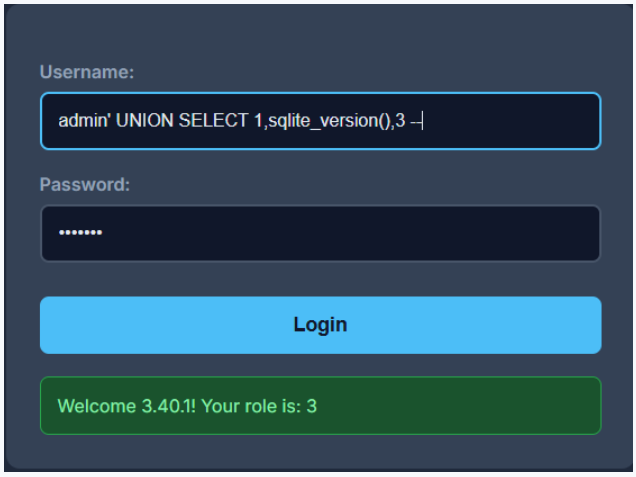
Three result columns / SQLite identification

The UNION probe succeeds with three values. The response displays the second value as the username and the third as the role, identifying useful output positions for data extraction.

```
admin' UNION SELECT 1,2,3 --
```



06. UNION result: username 2, role 3.



07. Database version returned in the page.

Identify the database

```
admin' UNION SELECT 1,sqlite_version(),3 --
```

The displayed result is **3.40.1**, identifying SQLite. Placing sqlite_version() in the second result column causes its output to appear in the welcome message.

Why UNION works here

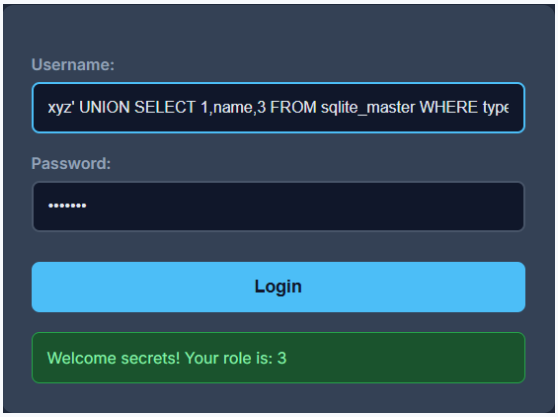
The injected SELECT must match the original query’s result-column count. Once a compatible result is returned, an expression in a displayed column can expose database information through the application response.

04 / DATABASE ENUMERATION

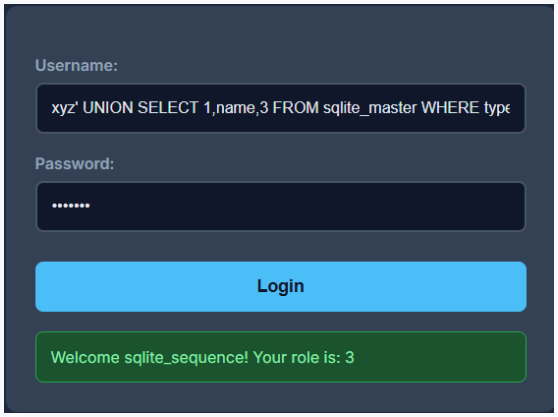
Tables and schema

Using sqlite_master to inspect database objects

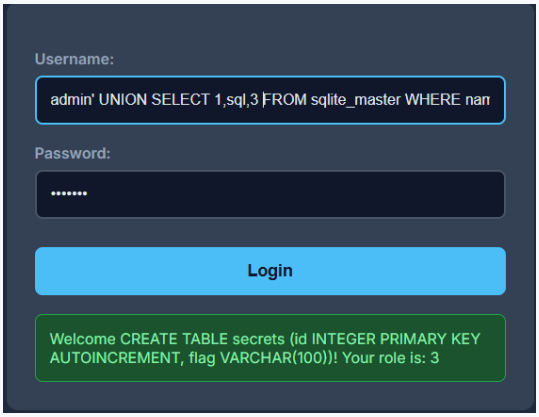
The walkthrough queries **sqlite_master** to discover tables, then reads stored CREATE TABLE statements. The screenshots identify **secrets** and **sqlite_sequence**. Some payload endings are clipped in the original images.



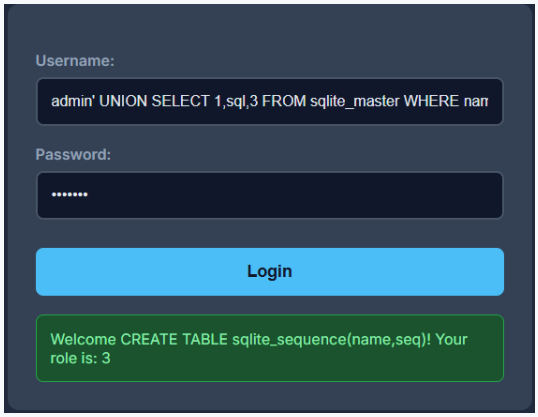
08. Table discovered: secrets.



09. Table discovered: sqlite_sequence.



10. Schema of the secrets table.



11. Schema of sqlite_sequence.

The field that matters

```
CREATE TABLE secrets (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  flag VARCHAR(100)
)
```

The schema reveals the **flag** column in **secrets**, providing the target for the final UNION query.

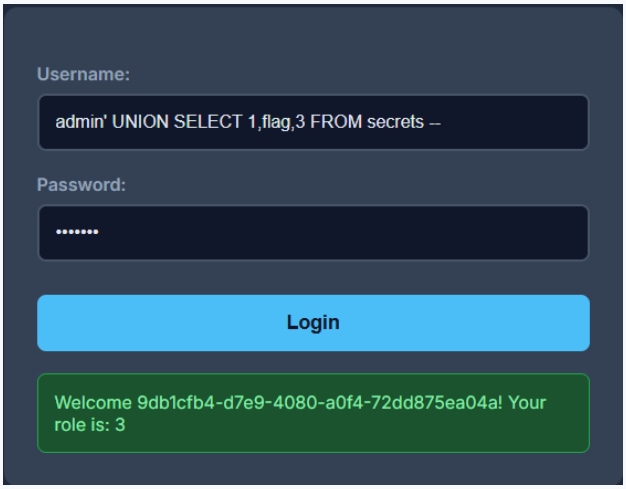
05 / FLAG AND LESSONS

Extracting the flag

From schema discovery to sensitive-data exposure

```
admin' UNION SELECT 1,flag,3 FROM secrets --
```

The flag is selected into the second output column, where the application displays it in the welcome message.



12. Original screenshot confirming flag extraction.

Impact

The recorded evidence demonstrates authentication bypass, database metadata discovery, and flag disclosure. Similar SQL injection flaws can expose credentials or other sensitive records; the full impact depends on database privileges and application design.

Conclusions and lessons learned

The exercise progresses from service enumeration to login-input testing, UNION-based extraction, SQLite schema inspection, and flag recovery. A vulnerable authentication query can expose much more than login functionality.

Use **parameterized queries**, enforce least-privilege database access, and validate inputs as an additional control. Avoid constructing SQL statements by concatenating untrusted input.

Edition note: English adaptation of the supplied PDF. All 12 screenshots are retained. Text has been clarified where it conflicted with the evidence. Commands were not re-executed for this edition.